

The system as built

A companion to *Pedestrian detection with the A121 radar* (21 September 2026). That document's sections 1 to 4 - the idea, the data, the feature extraction and the training - are unchanged and still correct. Its section 5 described an embedded pipeline that had not been written, on a toolchain that was not installed, over a link that was not decided. All three now exist and have run. This records what was built, what it measures, and where the specification turned out to be wrong.

1 Status

Stage	State	Where
Feature extraction (Python)	Done. 8 unit tests pin its behaviour	training/features.py
Binary model	Done. Trained, evaluated as int8, exported	training/out/final/
XM125 firmware	Done. Streams raw frames, 65 KB of 128 KB flash	firmware/xm125_streamer/
Feature extraction (C)	Done. Matches Python to 8e-6 dB on device	firmware/radar_infer/
Inference on device	Done. ~32 ms per window, 24 % of one core	firmware/radar_infer/
Display	Done. 480x480, status and scrolling spectrogram	firmware/radar_infer/

2 The shape of the built system

The radar module senses and streams; the ESP32-S3 computes and displays. Nothing else is in the loop - no PC, no protocol library, no handshake.

```

A121 --SPI-- XM125 (Cortex-M4, STM32L431CB)
    | raw Sparse IQ frames, 5632 B, 25 fps
    | UART 2 Mbaud, 141 kB/s, ONE WIRE + GND
    v
ESP32-S3  frame -> column -> 64-column window
          -> int8 CNN -> CLEAR / PEDESTRIAN
          -> 480x480 panel

```

Why the module does no signal processing. Moving the spectrogram onto the M4 would cut the link to 1.6 kB/s, but it would clone features.py into a second toolchain, and that copy would need its own proof of agreement with the Python. The DSP that already matches NumPy exactly stays in one place. The module stays deliberately dumb.

Why inference cannot live on the module. Arithmetic, not preference. The STM32L431CB has 128 KB of flash and 64 KB of RAM; the stock exploration server already occupies 109,168 bytes of that flash, 83 %. TensorFlow Lite Micro's runtime alone is several times what is left. The ESP32-S3 boots with 388 KB of free internal heap.

3 The link is one-directional, and that is deliberate

Only UART_TX and GND are wired. The ESP32's transmit pin is left unconnected because the XE125's own CP2105 USB bridge already drives that net, and the bridge must stay powered - the module needs a 1.8 V rail the jumper does not carry. Two transmitters on one net is contention.

Three consequences follow, and each one bit during bring-up:

Consequence	Why
-------------	-----

The firmware streams autonomously from boot	It can never be told to start. The exploration server's own startup waits for the host to release the RX line; copying that would have produced firmware that never starts.
Hardware flow control is disabled	The module would otherwise pause when CTS is de-asserted - and with no process holding the bridge's port open, the driver may leave it de-asserted for ever.
PC captures at 2 Mbaud are lossy by design	About a fifth of every frame, in 256-byte USB chunks. Build with <code>FLOW_CONTROL=1</code> when a complete capture is needed.

4 The wire format

```
A1 21 F0 0D | 64 sweeps x 22 range points of { int16 real, int16 imag }
              = 4 + 5632 bytes, 25 times a second
```

The magic word exists only so a receiver that loses synchronisation recovers on the next frame instead of emitting garbage indefinitely; the sensor provides no natural delimiter. $64 \times 22 \times 4 \times 25 = 141 \text{ kB/s}$ against a 2 Mbaud ceiling, about 77 % utilised. Measured: 141.9 kB/s, 25.0 fps, every frame intact. There is no headroom for a higher frame rate, which is one more reason the configuration stays frozen.

`tests/test_firmware_protocol.py` fails if the two firmwares and `tools/replay_frames.py` ever stop agreeing on this format, or if the C configuration stops matching `config/session_config.json`.

5 What it measures

Numbers from the device, not from a model of it.

Measurement	Result	Meaning
Boot self-test vs features.py	8e-6 dB worst, 0/64 int8 differ	The C port reproduces the Python exactly
Test vectors replayed	q=127 and q=-125, both exact	End to end agreement with the trained model
DSP	1.55 ms per frame	4 % of the 40 ms frame budget
Inference	~32 ms per window	20 % of the 160 ms window budget
Tensor arena	48,668 B measured	The specification guessed ~25 KB
Live empty room	124 windows, 0 false positives	Max p 0.035 against a 0.5 threshold
Live walk	98 of 110 windows positive	Detection 0.3 s after the first positive

The empty-room run matters more than the walk. Until it was made, the detector had only ever been shown to reject *recorded* backgrounds.

6 The display

480x480, driven directly rather than through LVGL: the board support package's own benchmark puts LVGL near 97 % CPU, and the detector needs a quarter of a core. Writing one column per frame costs a 640-byte transfer.

```
y  0.. 99  CLEAR / PEDESTRIAN, probability bar, threshold tick
y 100..419 spectrogram: 64 Doppler bins x 5 px, 480 columns = 19.2 s
y 420..479 APPROACH / DEPART axis
```

The spectrogram draws the int8 values *after* normalisation and quantisation - the literal bytes handed to the network. A separately scaled view would look better and would hide exactly the failure worth seeing. The cursor crosses in 19.2 s: that is the 25 Hz frame rate, not the 5500 Hz sweep rate, which resolves velocity within each frame.

7 Where the specification was wrong

Section 5 said	It is actually
Arena roughly 25 KB of activations	48,668 B, measured and logged at startup
Flashed over USB DFU	The STM32 UART bootloader. The button drives BOOT0; the module's own USB never reaches the XE125 connector
The XM125 is a Cortex-M33	A Cortex-M4, STM32L431CB
The ESP32 reads UART0 / GPIO44	UART1 on GPIO4. On the LCD-EV-Board almost every pin is taken by the panel; GPIO4 drives only the status LED
Inference cost unknown	~32 ms, and it rose from 26.4 ms when the display was added - the RGB panel competes for PSRAM

8 What is still not true

All 230 recordings come from one room. The project brief asks for two locations and that requirement is unmet. Claim generalisation to a new *person* - the evaluation is leave-one-subject-out - and not to a new *place*. A corridor test showed the detector still works in a geometry it has never seen, which is encouraging and is not evidence.

It is a motion detector at low SNR. The campaign recordings sit close to the noise floor and show no static clutter at all, unlike an earlier pilot with the same configuration. Both false alarms in the leave-one-subject-out evaluation were doors. No fan or trolley in the dataset was ever visible to it.

The highest-value next work is a second location, recorded under the same frozen configuration - not a different architecture.

9 Where everything is

Path	What
firmware/xm125_streamer/	The module firmware and its build script
firmware/radar_infer/	The ESP32-S3 firmware: DSP, inference, display
training/features.py	The reference DSP - the contract with the C
training/out/final/	Model, int8 tflite, C array, constants, test vectors
tools/replay_frames.py	Replays recordings or captures into the device
HARDWARE_SETUP.md	Build, flash, wire, verify - and the traps
TOOLCHAIN_SETUP.md	Why each tool is the one it is

Generated by tools/build_pipeline_addendum.py. The first document's generator was lost; this one is in the repository so that cannot happen again.